

Mintry Fabric

Mandate System Specification

Version 1.0

Status: Product & Engineering Specification

Purpose

This document defines the **Mandate System**, the core governance model of Mintry Fabric.

A Mandate is the primary object used to govern autonomous AI systems. It represents a runtime governance contract that defines the financial, operational, security, and compliance boundaries under which an AI agent is allowed to execute.

This terminology is used consistently across:

- Product UI
- SDK
- Dashboard
- Documentation
- Marketing
- Enterprise documentation

Internally, the platform uses policy evaluation and policy distribution mechanisms, but customers interact exclusively with **Mandates**.

Core Philosophy

Every autonomous AI agent must operate under a Mandate.

A developer should initialize Mintry once.

After initialization:

- no code changes,
- no redeployments,
- no configuration files,

should be required to change how an agent is governed.

All governance is managed through the Mintry Control Plane and distributed automatically to runtime interceptors.

What is a Mandate?

A **Mandate** is a versioned governance contract that defines how an AI agent is permitted to operate.

It specifies:

- financial limits
- approved AI providers
- approved models
- runtime restrictions
- compliance requirements
- operational controls

Every outbound AI request is evaluated against the active Mandate before execution.

Product Hierarchy

The hierarchy throughout the platform is:

Organization

↓

Projects

↓

Agents



Mandates



Runtime Rules

Example

Acme Corporation

Customer Support

Refund Agent

Mandate v4

Budget

Providers

Models

Runtime Controls

Compliance

Relationship Between Agents and Mandates

An Agent performs work.

A Mandate governs that work.

Customer Support Agent

↓

Mandate

↓

Runtime Evaluation

↓

Decision

↓

Provider

An Agent never executes without an active Mandate.

What a Mandate Contains

Identity

Name:

Customer Support Production

Version:

v3

Status:

Active

Owner:

Platform Engineering

Fiscal Controls

Controls AI spending.

Examples

Daily Budget:

\$500

Monthly Budget:

\$12,000

Maximum Request Cost:

\$0.25

Maximum Workflow Cost:

\$10

Provider Controls

Providers:

✓ OpenAI

✓ Anthropic

✗ Gemini

Model Controls

Allowed Models

GPT-4.1

Claude Sonnet

Claude Haiku

Runtime Controls

Maximum Requests Per Minute

200

Maximum Tokens

40,000

Maximum Runtime

30 seconds

Maximum Retries

3

Compliance Controls

PII Detection

Enabled

Prompt Logging

Disabled

Region

EU Only

Audit Required

Yes

Emergency Controls

Emergency Stop

Enabled

Auto Block

Enabled

Fail Closed

Disabled

Runtime Lifecycle

Step 1

Developer installs Mintry.

```
import mintry
```

```
mintry.init()
```

Step 2

Application registers with the Control Plane.

Step 3

Dashboard creates the first Mandate.

Customer Support

↓

Issue Mandate v1

Step 4

Runtime downloads the signed Mandate.

Dashboard



Mandate Distribution



Sidecar



Local Cache

Step 5

Every request evaluates locally.

Request



Mandate Evaluation



Allow

or

Block

Mandate Revisions

Mandates are immutable revisions.

Example

Mandate

v1



v2



v3



v4

Every revision contains:

- timestamp
- author
- changes
- approval status

Example

Revision

v7

Budget

\$500/day



\$300/day

Issued By

Jane Smith

Reason

Finance Optimization

Issuing a Mandate

Users never "edit policies."

Instead they issue a new Mandate revision.

Workflow

Edit

↓

Review

↓

Issue Mandate

↓

Sign

↓

Distribute

↓

Applied

Runtime Evaluation

Every outbound AI request follows:

Agent

↓

Request



Mandate Evaluation



Allowed?



YES



Provider

If not allowed

Agent



Request



Mandate Evaluation



Violation



Kill Switch



Blocked



Mandate Synchronization

Mandates are synchronized asynchronously.

Runtime execution never waits for the dashboard.

Workflow

Dashboard



Issue Mandate



Version 12



Sidecar Downloads



Validate Signature



Atomic Swap



Mandate Active

Offline Behaviour

If the Control Plane is unavailable

Runtime continues using

Last Verified Mandate

This guarantees

- zero additional latency
 - no dependency on Mintry Cloud
 - uninterrupted protection
-

Dashboard Terminology

Correct

Agents

Mandates

Issue Mandate

Mandate Revision

Mandate Ledger

Mandate Synchronization

Mandate Violations

Runtime Decisions

Governance

Headroom

Protected Spend

Blocked Requests

Avoid

Policy

Configuration

Settings

Rules

Budget Object

These remain internal implementation concepts.

Internal Architecture Terminology

Engineering terminology remains:

Policy Engine

Policy Compiler

Policy Evaluator

Policy Distribution

Policy Signature

Policy Bundle

Policy Cache

Policy Version

Customers never see these names.

UI Language Guide

Buttons

☒ Create Mandate

☒ Issue Mandate

☒ Duplicate Mandate

 Archive Mandate

 View Revisions

Status

Draft

Pending Review

Issued

Active

Superseded

Archived

Emergency

Events

Mandate Issued

Mandate Updated

Mandate Synchronized

Mandate Applied

Mandate Blocked Request

Mandate Violation

Emergency Mandate Activated

Future Dashboard Structure

Organization

Projects

Agents

Mandates

Runtime

Ledger

Telemetry

Audit

Billing

Design Principles

1. Initialize Once

Developers integrate Mintry a single time.

2. Govern Centrally

All runtime governance is authored through the Mintry Control Plane.

3. Enforce Locally

Every runtime decision is evaluated locally.

No request depends on the availability of Mintry Cloud.

4. Synchronize Asynchronously

Telemetry and Mandates synchronize in the background without affecting request latency.

5. Version Everything

Every Mandate is versioned, signed, auditable, and reversible.

6. Protect by Default

If connectivity is lost, Mintry continues enforcing the last verified Mandate.

The Mintry Definition

A Mandate is a versioned, signed governance contract that defines the operational boundaries under which an autonomous AI agent is permitted to execute.

Long-Term Vision

The Mandate System should become synonymous with Mintry in the same way that:

- **Pods** are synonymous with Kubernetes.
- **Commits** are synonymous with Git.
- **Providers** are synonymous with Terraform.

The goal is for engineering teams to naturally ask:

"Has this agent been issued a Mandate?"

rather than:

"Have we configured its policies?"

If Mintry consistently uses this language across its product, documentation, SDK, website, and enterprise onboarding, **"Mandate"** can become a distinctive concept that reinforces the product's identity and differentiates it from generic policy-based governance tools.